

WK 系列 UART 拓展 4 串口驱 动移植手册

修订记录

版本号	修订时间	作者	修订内容
V1.0	20200610	胥勋伟	创建版本

目录

修订记录	2
目 录	3
1 概述	错误!未定义书签。
2 封装	4
2.1 管脚列表	错误!未定义书签。
2.2 封装图	错误!未定义书签。
3 参考电路	错误!未定义书签。
4 特别申明	14

1 概述

1.1 WK 系列拓展芯片简介

目前 WK 系列能实现 UART 扩展 uart 的芯片包括 WK2114、WK2204、WK2168、WK2132、WK2212。目前 WK2114、WK2204、WK2168 能实现 uart 扩展 4 路 UART，WK2132、WK2212 能实现扩展 2 路 UART。目前这几款芯片使用的都是相同的 linux 驱动。

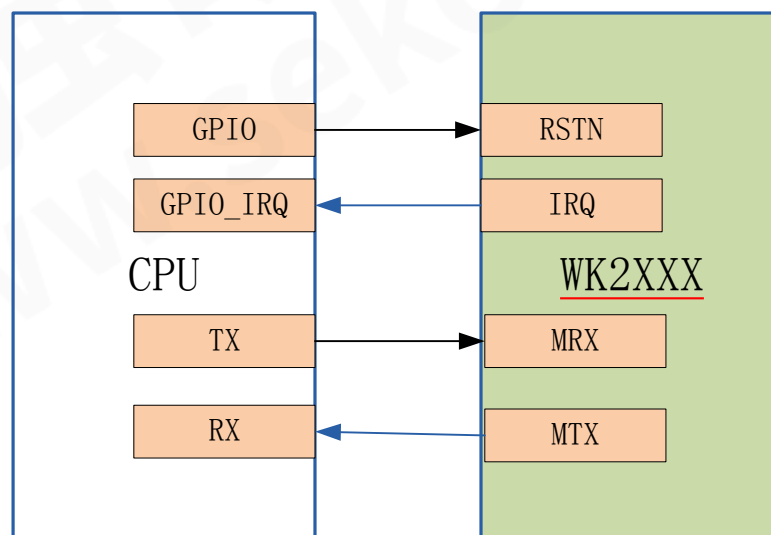
WK 系列扩展的子通道的 UART 具备如下功能特点：

每个子通道 UART 的波特率、字长、校验格式可以独立设置，最高可以提供 2Mbps(Vcc=5V)的通信速率。

每个子通道具备收/发独立的 256 级 FIFO(256 字节缓存)，FIFO 的中断可按用户需求进行编程触发点且具备超时中断功能。

2 封装

2.1 硬件连接示意图



- 1、WK 芯片作 UART 设备和 CPU 端的 UART 需要连接的信号 TX 和 RX 信号。CPU 的串口发送引脚 TX 连接到 WK 芯片的 MRX 引脚，CPU 串口的接收引脚 RX 连接到 WK 芯片的 MTX 引脚。
- 2、IRQ 信号为 WK 芯片的中断输出信号，需要连接到 CPU 具有外部中断功能的 GPIO 上，如图连接到 GPIO_IRQ。WK 芯片的复位信号引脚 RSTN，需要 CPU 用 GPIO 控制。

2.2 CPU 与 WK 芯片串口通信协议简介

微控制器和 WK 芯片的通信方式是控制字+数据的方式，命令通常包含了操作类型、子串口号、WK 芯片寄存器地址组成。具体的定义如下：

9.3.1. 写寄存器

UART	控制字节 CMD								1 个数据字节 DB(下行)							
BIT	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TX	0	0	C1	C0	A3	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
RX																

9.3.2. 写FIFO: (多字节写入)

UART	控制字节 CMD								[N3 N2 N1 N0]个数据字节 DB(下行)							
BIT	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TX	1	0	C1	C0	N3	N2	N1	N0	D7	D6	D5	D4	D3	D2	D1	D0
RX																

9.3.3. 读寄存器

UART	控制字节 CMD								1 个数据字节 DB(上行)							
BIT	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TX	0	1	C1	C0	A3	A2	A1	N0								
RX									D7	D6	D5	D4	D3	D2	D1	D0

9.3.4. 读FIFO: (多字节读取)

UART	控制字节 CMD								[N3 N2 N1 N0]个数据字节 DB(上行)							
BIT	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TX	1	1	C1	C0	N3	N2	N1	N0								
RX									D7	D6	D5	D4	D3	D2	D1	D0

说明：

- 1、 C1,C0: 子串口通道号, 00~11分别对应子串口1到子串口4; C1C0也可以表示全局寄存器的bit5, bit4。
- 2、 A3,A2,A1,A0:子串口寄存器地址;
- 3、 全局寄存器的6位地址对应为C1C0A3A2A1A0;
- 4、 N3,N2,N1,N0:写入/读取FIFO的数据字节个数;当其为0000时, 表明后接1个数据字节;当其为1111时, 表明后接16个数据字节;
- 5、 向子串口读/写数据有两种方法:
 - a) 读/写寄存器方式, 对子串口FIFO寄存器FDAT (1111) 进行读/写操作, 一次只能读/写一个字节
 - b) 读/写FIFO方式, 对接收/发送FIFO直接进行读/写操作, 一次最多可以读写16个连续数据

2.3 通信时序

下面的内容来源于WK2114的数据手册,

写操作时，先向WK2114的MRX写入一个命令字节 (Command Byte) , 随后写入相应的数据字节，其操作时序 (无校验模式) 如图9. 2. 1所示：

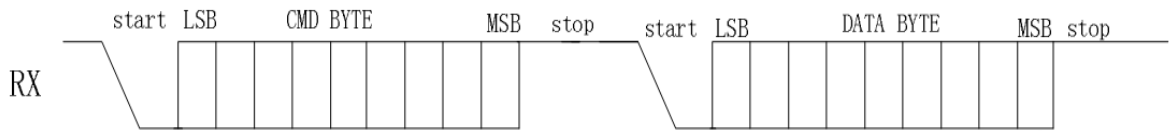


图 9. 2. 1 UART 主接口写操作时序

读操作时，先向WK2114的RX写入命令字节，相应的数据字节从MTX读取，其操作时序 (无校验模式) 如图9. 2. 2所示

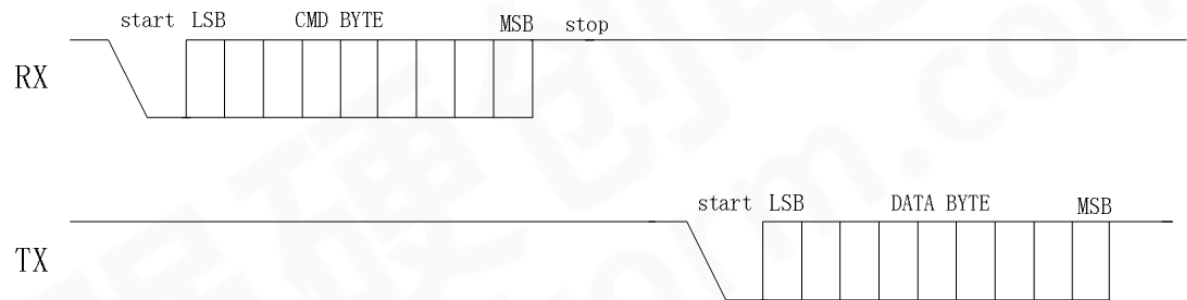


图 9. 2. 2 UART 主接口读操作时序

通信时序解读：

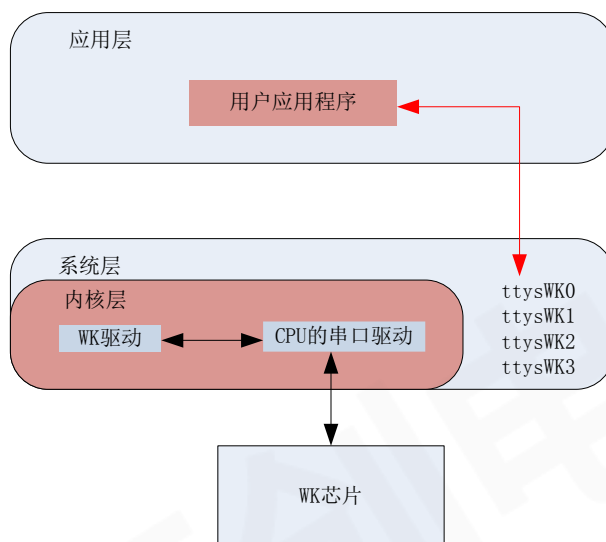
- 1、控制字节 CMD+DATA,先发送控制字节，然后发送数据或者接收数据。
- 2、特别注意，在接收数据期间，不能发送控制字节，否者芯片不能解析控制字节而导致时序紊乱。
- 3、多条操作同时执行时，一定需要按照顺序依次执行。

图 1WY16 系列 SOP8 封装尺寸

3 Linux 驱动

3.1 Linux 驱动基本框架简介

WK 系列 串口扩展芯片驱动遵循标准的串口驱动。驱动架构示意图如下：



1、WK 驱动工作在 linux 内核层，向上提供 4 个串口设备节点供应用层用户调用。也就是说 WK 驱动注册成功以后，在/dev/目录下会生成 ttysWK0、ttysWK1、ttysWK2、ttysWK3 共 4 个串口设备节点，应用层就可以按照操作普通串口节点的方式操作。

2、WK 驱动需要和 WK 芯片进行数据交互，数据交互是通过 CPU 的 UART 进行的，所以 WK 驱动会调用 CPU 的 UART 相关接口来完成数据传输。

4 驱动移植

4.1 驱动移植前的准备工作

4.1.1 硬件平台简介

目前串口驱动是在 AIO-3399J 开发板上开发和完成调试。该开发板上本身使用的是 WK2124 这颗芯片，实现了 SPI 转 4 串口。现在我把 WK2124 这颗芯片换成了 WK2114，实现了 UART 转 4 串口。硬件上也做了一些改动，这部分大家可以不必太关注。

4.1.2 软件平台简介

目前我们在 AIO-3399J 上安装的是 ubuntu 的系统，刷的是 ubuntu 的固件，内核是 4.4。

4.1.3 驱动文件介绍

驱动版本是 V2.1

驱动为文件包括: wk2xxx_uart.c wk2xxx.h Makefile

wk2xxx_uart.c 驱动程序源文件

wk2xxx.h 驱动程序头文件

Makefile makefile 文件

4.2 驱动移植

目前我们对 WK 串口芯片和相关驱动有了一定的了解,那我们下一步,就来看看怎么移植驱动。

4.2.1 配置 DTS 文件

为什么需要配置 DTS 和怎么配置 DTS 那?下面对这些问题做一些简单的回答:

- 1、DTS 是描述系统信息的数据结构或者是系统资源,如果 WK 驱动需要从系统获取相关的资源,那么就需要先配置好 DTS,然后在驱动中才能获取到。
- 2、怎么配置 DTS 或者说我们根据什么原则来配置 DTS,其实就是根据我们需求来决定。比如 WK 串口拓展 4 串口,从硬件电路上来看,我们需要 cpu(系统)提供一个 UART,一个复位的 GPIO,一个中断 GPIO,那么我们就需要在 DTS 中把相关的信息给出来。下面来看看具体的 DTS 配置

首先我们来看 rk3399.dtsi 中对 uart4 的描述,包含 uart4 基本配置

```
uart4: serial@ff370000 {
    //compatible = "rockchip,rk3399-uart", "snps,dw-apb-uart";
    reg = <0x0 0xff370000 0x0 0x100>;
    clocks = <&pmucru SCLK_UART4_PMU>, <&pmucru PCLK_UART4_PMU>;
    clock-names = "baudclk", "apb_pclk";
    interrupts = <GIC_SPI 102 IRQ_TYPE_LEVEL_HIGH 0>;
    reg-shift = <2>;
    reg-io-width = <4>;
    pinctrl-names = "default";
    pinctrl-0 = <&uart4_xfer>;
    status = "disabled";
};
```

注意: WK 驱动用的是 RK3399 的 uart4 来作为扩展串口,所以我们在 dts 中设置的 uart4. 这个根据自己的实际使用来选择。

再看 dts 中的配置：

```

120 &uart4 {
121     // status = "disabled";
122     //兼容性属性，通常用来做总线匹配，和驱动中该属性保持一致
123     compatible = "rockchip,wk2xxx_uart";
124     //复位引脚，
125     reset-gpio = <&gpio1 11 GPIO_ACTIVE_HIGH>;
126     //中断引脚。
127     irq-gpio = <&gpio1 12 IRQ_TYPE_LEVEL_LOW>;
128     //uart4波特率设置
129     rockchip,baudrate = <500000>; /* Only 115200 and 1500000 */
130
131     status = "okay";
132 };
133

```

DTS 中的配置如上，我们来分析一下

1、compatible 该项作为匹配项，需要和驱动中保持一致，如果下图为驱动中相同项。

```

2162 */
2163 static const struct of_device_id wk2xxx_of_match[] = {
2164     { .compatible = "rockchip,wk2xxx_uart" },
2165     {}
2166 };
2167

```

只有保持一致，驱动才能加载成功，才能进去驱动的 probe 函数，如下：

```
static int wk2xxx_probe(struct platform_device *pdev)
```

2、reset-gpio 是 WK 芯片复位的控制引脚，

3、irq-gpio 是和 WK 芯片 IRQ 引脚连接的中断引脚，特别注意的是触发模式需要配置成低电平触发，也就是 IRQ_TYPE_LEVEL_LOW

4、其他配置项，根据实际需求修改

现在 DTS 端基本上设置完成，编译以后重新更新到开板。那么下面就可以开始着手驱动修改和移植了。

4.2.2 驱动分析与修改

首先我们要明白一个问题，既然我们要修改驱动，那么到底要修改那些。通常来说，主要是修改平台差异相关的和硬件相关的，就 WK 的驱动来看，主要就是修改和 cpu 串口相关的；由于复位和中断引脚相关的在 DTS 中基本上都配置好了，在驱动中基本上就不用修改。

那么我们就简单介绍一下,怎么修改cpu串口相关的,修改驱动源文件 wk2xxx_uart.c。

4.2.2.1 CPU 串口的初始化

- 1、首先来看 wk2xxx_probe(struct platform_device *pdev)函数, 该函数在驱动加载的时候, 如果总线 (UART 接口) 匹配成功调用该函数。该函数主要实现了如下的一些功能:

首先是从 DTS 获取相关的资源信息, 比如 CPU 串口 uart4 的信息并初始化 UART4 如下图: 由于代码量比较多, 并没有截取完整。

```

1925     printk(KERN_ALERT "%s!--in--\n", __func__);
1926
1927     g_uart_port = devm_kzalloc(&pdev->dev, sizeof(*g_uart_port), GFP_KERNEL);
1928     if (!g_uart_port)
1929     {
1930         printk(KERN_ALERT "%s!--devm_kzalloc ----fail--\n", __func__);
1931         return -ENOMEM;
1932     }
1933
1934     uart_resource = platform_get_resource(pdev, IORESOURCE_MEM, 0);
1935     //printk(KERN_ALERT "%s!--uart_resource :%x--,%srt:%x;end:%x;name:%s\n", __func__,uart_resource,uart_resource->start,resource_size(uart_resource),uart_resource->name);
1936     devm_request_mem_region(&pdev->dev, uart_resource->start, resource_size(uart_resource),uart_resource->name);
1937     printk(KERN_ALERT "%s!--ret :%x--\n", __func__,ret);
1938     g_uart_port->port.membase = ioremap(uart_resource->start, uart_resource->end - uart_resource->start + 1);
1939     printk(KERN_ALERT "%s!--g_uart_port ->port.membase :%p--\n", __func__,g_uart_port ->port.membase);
1940
1941
1942     irq = platform_get_irq(pdev, 0);
1943     if (irq < 0)
1944     {
1945         dev_err(&pdev->dev, "no irq resource?\n");
1946         return irq;
1947     }
1948     printk(KERN_ALERT "%s!-- platform_get_irq;%d--\n", __func__,irq);
1949
1950
1951     #ifdef CONFIG_OF
1952     of_rk_serial_parse_dt(pdev->dev.of_node, &rks);
1953     pdev->id = rks.id;
1954     #endif
1955     printk(KERN_ALERT "%s!--rk3399_serial.%d, pdev->id--\n", __func__,pdev->id);
1956     g_uart_port->pdev = pdev;
1957
1958     g_uart_port->tx_loadsz = 30;
1959     g_uart_port->port.dev = &pdev->dev;
1960     g_uart_port->port.type = PORT_8250;
1961     g_uart_port->port.irq = irq;
1962     g_uart_port->port.iotype = UPIO_MEM;
1963     g_uart_port->port.regshift = 2;
1964     //fifo size default is 32, but it will be updated later when start_up
1965     g_uart_port->port.fifosize = 32;
1966     //g_uart_port->port.ops = &serial_rk_pops;
1967     g_uart_port->port.line = pdev->id;
1968     printk(KERN_ALERT "%s! g_uart_port->port.line=%d!!\n", __func__,g_uart_port->port.line);
1969     g_uart_port->port.line = DBG_PORT;

```

```

1977 g_uart_port->clk = devm_clk_get(&pdev->dev, "baudclk");
1978 if (IS_ERR(g_uart_port->clk) && PTR_ERR(g_uart_port->clk) != -EPROBE_DEFER){
1979     g_uart_port->clk = devm_clk_get(&pdev->dev, NULL);
1980 }
1981 if (IS_ERR(g_uart_port->clk) && PTR_ERR(g_uart_port->clk) == -EPROBE_DEFER){
1982     return -EPROBE_DEFER;
1983 }
1984 if (!IS_ERR_OR_NULL(g_uart_port->clk)) {
1985     err = clk_prepare_enable(g_uart_port->clk);
1986     if (err){
1987         dev_warn(&pdev->dev, "could not enable optional baudclk: %d\n",
1988             err);
1989     }
1990     else{
1991         g_uart_port->port.uartclk = clk_get_rate(g_uart_port->clk);
1992     }
1993 }
1994
1995 /* If no clock rate is defined, fail. */
1996 if (!g_uart_port->port.uartclk) {
1997     dev_err(&pdev->dev, "clock rate not defined\n");
1998     return -EINVAL;
1999 }
2000
2001 g_uart_port->pclk = devm_clk_get(&pdev->dev, "apb_pclk");
2002 if (IS_ERR(g_uart_port->pclk) && PTR_ERR(g_uart_port->pclk) == -EPROBE_DEFER) {
2003     err = -EPROBE_DEFER;
2004     goto err_clk;
2005 }
2006 if (!IS_ERR(g_uart_port->pclk)) {
2007     err = clk_prepare_enable(g_uart_port->pclk);
2008     if (err) {
2009         dev_err(&pdev->dev, "could not enable apb_pclk\n");
2010         goto err_clk;
2011     }
2012 }
2013 /*rk3399 uart4 init*/
2014 uart_rk_init(g_uart_port);
2015

```

由于这段代码，在不同的平台下面可能并不一样，那么我们怎么来实现这段带呀。通常我们可以参考使用平台下串口驱动中的 probe 函数来实现。具体需要实现的功能就是：申请 cpu 串口的使用资源、设置 cpu 串口的相关配置、初始化 cpu 串口

4.2.2.2 复位引脚和中断引脚获取

我们继续研究 `wk2xxx_probe(struct platform_device *pdev)` 函数。如下图所示：这个地方主要是从 DTS 中获取已经给出的中断和复位引脚。由于在 dts 中已经配置好了这两个 GPIO，所以这个地方就需要修改。如果出现编号无效或者其他信息，那么请检查 DTS 的配置是否有问题，或者是 gpio 是否已经被其他地方占用。如下图是获取 DTS 中的复位和中断引脚的信息。

```

wk_reset=rockchip_reset_gpio_init(&pdev->dev);
if(wk_reset<0)
{
    printk(KERN_ALERT "%s! reset_GPIO = 0x%d;reset_GPIO request fail!!!\n",__func__,wk_reset);
    return 1;
}
printk(KERN_ALERT "%s! reset = 0x%d;reset GPIO request sucess!!!\n",__func__,wk_reset);

wk_irq = rockchip_irq_gpio_dt(&pdev->dev);
if(wk_irq<0)
{
    printk(KERN_ALERT "%s! IRQ_GPIO = 0x%d;IRQ_GPIO request fail!!!\n",__func__,wk_irq);
    return 1;
}
printk(KERN_ALERT "%s! IRQ_GPIO = 0x%d;IRQ_GPIO request sucess!!!\n",__func__,wk_irq);

```

与之对应的 DTS 的配置如下图红色框

```

7  &uart4 {
8      //status = "disabled";
9      compatible = "rockchip.wk2xxx uart";
10     reset-gpio = <&gpio1 11 GPIO_ACTIVE_HIGH>;
11     irq-gpio = <&gpio1 12 IRQ_TYPE_LEVEL_LOW>;
12     rockchip,baudrate = <500000>;
13     status = "okay";
14 };
15

```

4.2.2.3 主串口波特率匹配与测试主串口通信

主串口自动波特率匹配的原理：

WK2114 的主串口在芯片复位以后，芯片会检测 MRX 引脚上电平的变化，把最短的低电平，作为一个 bit 位的时间长度，以后就按照这个波特率进行通信(除非硬件复位芯片，否者主串口的波特率是不能改变的)。

具体操作：复位引脚 wk_reset 拉低 10ms，然后再拉高，完成芯片的复位。然后 CPU 的串口向 MRX 引脚发送 0X55，WK2114 根据发送 0x55 时的波特率，锁定主串口波特率，波特率锁定以后，就可以进行读写 wk2114 的寄存器 GENA 了，默认值是 0XF0。

具体的实现代码如下图：

```

2034    udelay(100);
2035    /*reset wk2114*/
2036    mdelay(10);
2037    gpio_set_value(wk_reset, 1);
2038    mdelay(10);
2039    gpio_set_value(wk_reset, 0);
2040    mdelay(10);
2041    gpio_set_value(wk_reset, 1);
2042    mdelay(10);
2043    /**put 0x55 to wk2114**/
2044    uart_putc(g_uart_port,0x55);
2045    udelay(10);
2046
2047    do
2048    {
2049        wk2xxx_read_reg(g_uart_port,1,WK2XXX_GENA,dat);
2050        printk(KERN_ALERT "WK2XXX_GENA:0x%x\n",dat[0]);
2051    }while(0);
2052

```

复位芯片

发送0x55，匹配波特率

读GENA寄存器，测试主接

常见问题:

- 1、是否可以不用 GPIO 控制 WK2114 芯片复位，而是用上电复位的方式。

答：上电复位芯片的方式在实际应用中受到限制。通常 CPU 和 WK2114 上电复位以后。CPU 会进行初始化，包括初始化和 WK2114 连接的这个串口。在初始化的过程中，cpu 串口的 IO 电平可能会改变，那么这个变化可能会被 WK2114 当做波特率匹配，最终导致后面发送 0x55 的时候匹配失败。那么加上 GPIO 复位以后，就排除 cpu 串口初始化的干扰，保证了波特率匹配能成功。

4.2.2.4 注册设备驱动和串口设备节点

- 1、如下图所示

```

mutex_lock(&wk2xxxs_lock);
if(!uart_driver_registered)
{
    uart_driver_registered = 1;
    status1=uart_register_driver(&wk2xxx_uart_driver);
    if (status1)
    {
        printk(KERN_ERR "Couldn't register wk2xxx uart driver\n");
        mutex_unlock(&wk2xxxs_lock);
        return status1;
    }
}

printk(KERN_ALERT "wk2xxx_serial_init()\n");

for(i =0;i<NR_PORTS;i++)
{
    struct wk2xxx_port *s = &wk2xxxs[i]; //container_of(port,struct wk2xxx_port,port);
    s->tx_done =0;
    s->port.line = i;
    s->port.ops = &wk2xxx_pops;
    s->port.uartclk = WK_CRASTAL_CLK;
    s->port.fifosize = 256;
    s->port.iobase = i+1;
    s->port.irq = wk_irq;
    s->port.iotype = SERIAL_IO_PORT;
    s->port.flags = ASYNC_BOOT_AUTOCONF;
    status1 = uart_add_one_port(&wk2xxx_uart_driver, &s->port);
    if(status1<0)
    {
        printk(KERN_ALERT "uart_add_one_port failed for line i:= %d with error %d\n",i,status1);
    }else{
        printk(KERN_INFO "uart_add_one_port success for line i:= %d with right %d\n",i,status1);
    }
}

printk(KERN_ALERT "uart_add_one_port = 0x%d\n",status1);
mutex_unlock(&wk2xxxs_lock);

```

5 特别申明

- 成都为开微电子有限公司（简称为开微电子）保有在不事先通知而修改这份文档的权利。在使用本产品前请联系为开微电子获取该新产品说明的最新版本。
- 为开微电子认为提供的信息是准确可信的。尽管这样，为开微电子对文档中可能出现的错误不承担任何责任。
- 对于使用该器件引起的专利纠纷及第三方侵权为开微电子不承担任何责任。
- 为开微电子的产品不建议应用于生命相关的设备和系统，在使用该器件中因为设备或系统运转失灵而导致的损失为开微电子不承担任何责任。
- 为开微电子对本手册拥有版权等知识产权，受法律保护。未经为开微电子许可，任何单位及个人不得以任何方式或理由对本手册进行使用、复制、修改、抄录、传播等。